

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерных технологий

Основы программной инженерии

Лабораторная работа №3

Вариант 40929

Выполнила:

Павличенко Софья Алексеевна, Р3215

Проверила:

Харитоновна Анастасия Евгеньевна

Санкт-Петербург 2025г.

Оглавление

Задание	3
build.gradle	4
pom.xml	6
Заключение	10

Задание

Написать сценарий для утилиты Maven и Gradle, реализующий компиляцию, тестирование и упаковку в jar-архив кода проекта из лабораторной работы №3 по дисциплине "Веб-программирование".

Каждый этап должен быть выделен в отдельный блок сценария; все переменные и константы, используемые в сценарии, должны быть вынесены в отдельный файл параметров; MANIFEST.MF должен содержать информацию о версии и о запуске класса.

Сценарий должен реализовывать следующие цели (targets):

1. **compile** -- компиляция исходных кодов проекта.
2. **build** -- компиляция исходных кодов проекта и их упаковка в исполняемый jar-архив. Компиляцию исходных кодов реализовать посредством вызова цели **compile**.
3. **clean** -- удаление скомпилированных классов проекта и всех временных файлов (если они есть).
4. **test** -- запуск junit-тестов проекта. Перед запуском тестов необходимо осуществить сборку проекта (цель **build**).
5. **scp** - перемещение собранного проекта по scp на выбранный сервер по завершению сборки. Предварительно необходимо выполнить сборку проекта (цель **build**).
6. **env** - осуществляет сборку и запуск программы в альтернативных окружениях; окружение задается версией java и набором аргументов виртуальной машины в файле параметров.

build.gradle

```
plugins {  
    id 'org.jetbrains.kotlin.jvm' version '1.9.23'  
    id 'war'  
}  
  
group = 'org.example'  
version = '1.0-SNAPSHOT'  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
  
    compileOnly 'jakarta.platform:jakarta.jakartaee-api:9.1.0'  
    compileOnly 'org.primefaces:primefaces:12.0.0'  
  
    implementation 'org.eclipse.persistence:org.eclipse.persistence.jpa:3.0.2'  
    implementation 'org.postgresql:postgresql:42.7.2'  
  
    testImplementation 'org.jetbrains.kotlin:kotlin-test'  
}  
  
war {  
    archiveFileName = warFileName  
  
    manifest {  
        attributes(  
            'Implementation-Title': project.name,  
            'Implementation-Version': project.version,  
        )  
    }  
}  
  
tasks.register('compile') {  
    group = 'build'  
    description = 'Compiles the source code'  
    dependsOn compileKotlin  
}  
  
tasks.register('myBuild') {  
    group = 'build'  
    description = 'Compiles the source code'  
    dependsOn compile, war, classes  
}  
  
clean {  
    group = 'build'  
    description = 'Deletes compiled classes and temporary files'  
}  
  
test {  
    group = 'verification'  
    description = 'Runs JUnit tests'  
    dependsOn myBuild  
    useJUnitPlatform()  
}
```

```

tasks.register('scp', Exec) {
    group = 'deployment'
    description = 'Deploys WAR via SCP'
    dependsOn myBuild

    def warFile = layout.buildDirectory.file("libs/${warFileName}")

    environment = ['SSHPPASS': password]
    commandLine = [
        'sshpass', '-e',
        'scp', '-P', port,
        warFile.get().asFile.absolutePath,
        "${user}@${host}:${destinationDir}"
    ]
}

tasks.register('env', Exec) {
    group = 'environment'
    description = 'Running in an alternative environment'
    dependsOn myBuild, scp

    commandLine = [
        'sshpass', '-e',
        'ssh', '-p', port,
        "${user}@${host}",
        'bash', '-c',
        "sh ${wildflyDir}/bin/standalone.sh"
    ]

    environment = [
        'SSHPPASS': password,
        'JAVA_HOME': pathToJava,
        'ALT_JVM_ARGS': altJvmArgs
    ]
}

kotlin {
    jvmToolchain(17)
}

```

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.example</groupId>
  <artifactId>lab3</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>war</packaging>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <kotlin.code.style>official</kotlin.code.style>
    <kotlin.compiler.jvmTarget>1.8</kotlin.compiler.jvmTarget>

    <war.file.name>server</war.file.name>
    <ssh.host>helios.cs.ifmo.ru</ssh.host>
    <ssh.user>s409295</ssh.user>
    <ssh.password>dfhA,0325</ssh.password>
    <ssh.port>2222</ssh.port>

<ssh.destination.dir>web/lab3/wildfly/standalone/deployments</ssh.destination.dir>
>
    <ssh.wildfly.dir>web/lab3/wildfly</ssh.wildfly.dir>
    <path.to.java>usr/local</path.to.java>
    <alt.java.version>17</alt.java.version>
    <alt.jvm.args>-Dfile.encoding=UTF-8</alt.jvm.args>
  </properties>

  <repositories>
    <repository>
      <id>mavenCentral</id>
      <url>https://repo1.maven.org/maven2</url>
    </repository>
  </repositories>

  <build>
    <sourceDirectory>src/main/kotlin</sourceDirectory>
    <plugins>
      <plugin>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-maven-plugin</artifactId>
        <version>1.9.23</version>
        <executions>
          <execution>
            <id>compile</id>
            <phase>compile</phase>
            <goals>
              <goal>compile</goal>
            </goals>
          </execution>
        </executions>
      </plugin>

      <plugin>
```

```

    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-war-plugin</artifactId>
    <version>3.4.0</version>
    <executions>
      <execution>
        <phase>package</phase>
        <goals>
          <goal>war</goal>
        </goals>
      </execution>
    </executions>
    <configuration>
      <warName>${war.file.name}</warName>
      <archive>
        <manifest>

<addDefaultImplementationEntries>true</addDefaultImplementationEntries>
        </manifest>
        <manifestEntries>
          <Implementation-
Title>${project.name}</Implementation-Title>
          <Implementation-
Version>${project.version}</Implementation-Version>
          <Build-Jdk>${java.version}</Build-Jdk>
        </manifestEntries>
      </archive>
    </configuration>
  </plugin>

  <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>2.22.2</version>
    <executions>
      <execution>
        <phase>test</phase>
        <goals>
          <goal>test</goal>
        </goals>
      </execution>
    </executions>
  </plugin>

  <plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>exec-maven-plugin</artifactId>
    <version>3.0.0</version>
    <executions>
      <execution>
        <id>scp</id>
        <phase>install</phase>
        <goals>
          <goal>exec</goal>
        </goals>
        <configuration>
          <executable>sshpass</executable>
          <arguments>
            <argument>-e</argument>
            <argument>scp</argument>
            <argument>-P</argument>
            <argument>${ssh.port}</argument>
          </arguments>
        </configuration>
      </execution>
    </executions>
  </plugin>

```

```

<argument>${project.build.directory}/${war.file.name}.war</argument>

<argument>${ssh.user}@${ssh.host}:${ssh.destination.dir}</argument>
    </arguments>
    <environmentVariables>
        <SSHPASS>${ssh.password}</SSHPASS>
    </environmentVariables>
</configuration>
</execution>

<execution>
    <id>env</id>
    <phase>install</phase>
    <goals>
        <goal>exec</goal>
    </goals>
    <configuration>
        <executable>sshpas</executable>
        <arguments>
            <argument>sshpas</argument>
            <argument>-e</argument>
            <argument>ssh</argument>
            <argument>-p</argument>
            <argument>${ssh.port}</argument>
            <argument>${ssh.user}@${ssh.host}</argument>
            <argument>bash</argument>
            <argument>-c</argument>
            <argument>' sh
${ssh.wildfly.dir}/bin/standalone.sh'</argument>
        </arguments>

        <environmentVariables>
            <SSHPASS>${ssh.password}</SSHPASS>
            <JAVA_HOME>${path.to.java}</JAVA_HOME>
            <ALT_JVM_ARGS>${alt.jvm.args}</ALT_JVM_ARGS>
        </environmentVariables>
    </configuration>
</execution>
</executions>
</plugin>

</plugins>
</build>

<dependencies>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-test-junit5</artifactId>
        <version>1.9.23</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-engine</artifactId>
        <version>5.10.0</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.jetbrains.kotlin</groupId>
        <artifactId>kotlin-stdlib</artifactId>

```

```
        <version>1.9.23</version>
    </dependency>

    <dependency>
        <groupId>jakarta.platform</groupId>
        <artifactId>jakarta.jakartaee-api</artifactId>
        <version>9.1.0</version>
        <scope>provided</scope>
    </dependency>
    <dependency>
        <groupId>org.primefaces</groupId>
        <artifactId>primefaces</artifactId>
        <version>12.0.0</version>
        <scope>provided</scope>
    </dependency>

    <dependency>
        <groupId>org.eclipse.persistence</groupId>
        <artifactId>org.eclipse.persistence.jpa</artifactId>
        <version>3.0.2</version>
    </dependency>

    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <version>42.7.2</version>
    </dependency>

</dependencies>

</project>
```

Заключение

В результате выполнения работы я ознакомилась с системами сборки Gradle и Maven, изучила их особенности и различия. На практике разобралась с основными концепциями работы с зависимостями, фазами жизненного цикла и плагинами в обеих системах. Реализовала сборку проекта с упаковкой в WAR-файл и выполнением дополнительных задач, таких как тестирование, развертывание на сервере и настройка окружений.